COMS W4771: Machine Learning

Lecture 22: Regularization in Neural Networks

Tony Dear, Ph.D.

Department of Computer Science

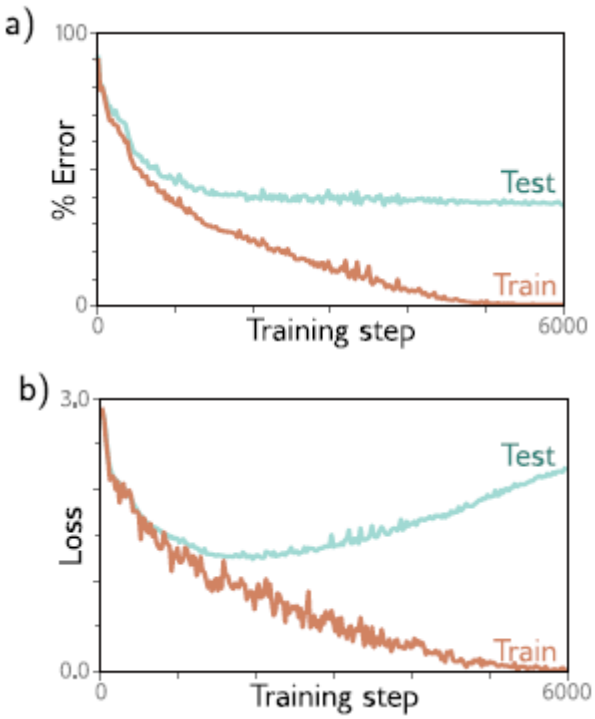
School of Engineering and Applied Sciences

Topics

- Overfitting and double descent
- Explicit and implicit regularization
- Early stopping, ensembles, and Bayesian inference
- Dropout, adding noise, and augmentation

Overfitting

- Neural networks are extremely complex models with a large number of parameters
- Very prone to *overfitting* the training data and achieving perfect performance
- With continued optimization, test error will decrease before eventually increasing again
- Model makes the same mistakes, but becomes increasingly confident in its wrong predictions



Prince

Bias-Variance Tradeoff

- As usual, expected test error is due to inherent noise, bias, and variance
- Variance can be reduced if we have access to more training data

- To reduce bias, we increase model flexibility by adding capacity
- Allows the predicted function to fit the true function more closely

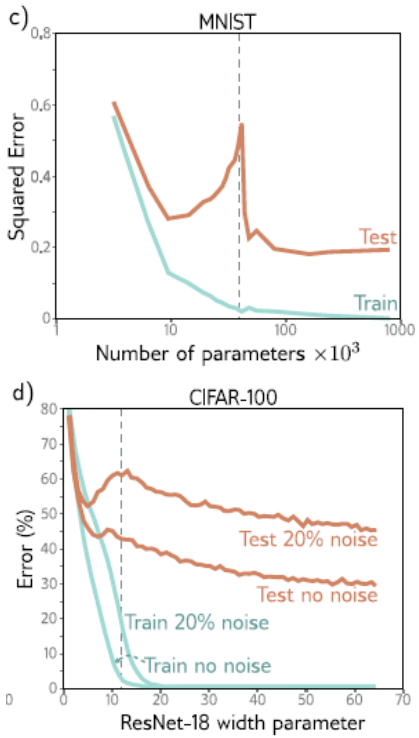
- Due to bias-variance tradeoff, increased model capacity will eventually start fitting noise in the training data, again increasing variance
- We will need to use regularization methods to systematically reduce variance

Double Descent

- Test error typically exhibits a U-shaped curve as a function of model complexity
- Decreases in the *under-parameterized* regime and increases in the *critical* regime (due to model variance)

- But in some settings, test error may decrease *again* in the *over-parameterized regime*

- Contradicts conventional understanding of overfitting
- Suggests that *large* models may perform fairly well



Prince

Double Descent

- The switch typically occurs near the *interpolation threshold* of the problem
- Number of parameters is just enough to fit the data perfectly on average

- Due to curse of dimensionality, training data can be very spread out
- The more underparameterized the model, the rougher the resultant function

- With more capacity, the model can start interpolating smoother functions *between* the training data, esp in high dimensions
- Smoother (simpler) functions may generalize better on unseen data

Explicit Regularization

- As with other models, we can add a penalty term to the loss function:

$$\hat{\boldsymbol{\phi}} = \underset{\boldsymbol{\phi}}{\operatorname{argmin}} \left(\sum_{i=1}^{N} l_i(\mathbf{y}_i, \mathbf{f}(\mathbf{x}_i)) + \lambda g(\boldsymbol{\phi}) \right)$$

- For example, g may be the L_1 (lasso) or L_2 norm of the weights $\boldsymbol{\phi}$
- As usual, we typically only penalize the weights $\boldsymbol{\Omega}$ but not the biases $\boldsymbol{\beta}$

- This is particularly important when the model is *over-parameterized*
- Since there are multiple solutions that can fit the data (near) perfectly, regularization pushes the model toward a smoother (simpler) one

Gradient Descent Regularization

- In gradient descent, each step moves along an *approximation* of $-\frac{\partial L}{\partial \boldsymbol{\phi}}$ since it is discrete and finite-sized, as opposed to infinitesimally small
- Instead of the original loss L, can show that the GD update $\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} - \alpha \frac{\partial L}{\partial \boldsymbol{\phi}}$ is actually minimizing

$$\tilde{L}_{GD}(\boldsymbol{\phi}) = L(\boldsymbol{\phi}) + \frac{\alpha}{4}\left\|\frac{\partial L}{\partial \boldsymbol{\phi}}\right\|^2$$

- L and $\tilde{L}_{GD}$ have the same minima, but $\tilde{L}_{GD}$ is penalized elsewhere when the gradient norm is large, pushing GD to prefer *flatter* regions of $\tilde{L}_{GD}$
- These regions tend to be more stable and better for generalization!

Stochastic Gradient Descent

- In stochastic gradient descent, we approximate the loss function due to both discrete updates as well as gradient estimates from data minibatches
- Can show that modified loss also depends on loss of each minibatch L_b:

$$\tilde{L}_{SGD}(\boldsymbol{\phi}) = L(\boldsymbol{\phi}) + \frac{\alpha}{4}\left\|\frac{\partial L}{\partial \boldsymbol{\phi}}\right\|^2 + \frac{\alpha}{4B}\sum_{b=1}^{B}\left\|\frac{\partial L_b}{\partial \boldsymbol{\phi}} - \frac{\partial L}{\partial \boldsymbol{\phi}}\right\|^2$$

- New regularization term penalizes the *difference* between each batch loss and overall loss gradient, so stable gradient regions are preferred

- This also suggests that *smaller* batch sizes yield more regularization, leading to better generalization and performance

Early Stopping

- Similar to bagging and boosting models, we can observe the loss on the training data as we train a neural network (e.g., loss per epoch)

- Stopping *before* convergence can help us avoid overfitting to noise
- Since weights are initialized as small values, they do not have *time* to become large, which implicitly regularizes them and reduces model complexity!

- If we have validation data or perform CV, we can simply stop around the training iteration when validation loss is at its lowest
- We can also store multiple models while training to compare them later

Ensembles

- Another way to reduce variance is to train an *ensemble* of NN models
- Models can have different initializations, architectures, and hyperparameters

- Unlike bagging trees, the trained models will vary even with the same data
- Each one can land in a different local optimum, yielding *diverse* models

- More generally, we can again apply bagging and train each model on a sample of the training data, which also gives us OOB evaluation
- We can average the pre-softmax outputs for the final prediction

Bayesian Inference

- Instead of pointwise estimation of the most likely parameters, an alternative is to learn and update a *distribution* over the parameters

- If $\Pr(\phi)$ is a *prior* over the parameters, then the *posterior* $f(\phi)$ can be computed using the *likelihood* $\Pr(\mathbf{y}|\mathbf{x}, \phi)$ of the training data and Bayes' rule:

$$f(\phi) = \Pr(\phi|\mathbf{x}, \mathbf{y}) \propto \Pr(\mathbf{y}|\mathbf{x}, \phi)\Pr(\phi)$$

- Then we can predict a new input by integrating or summing over ϕ:

$$\Pr(\mathbf{y}|\mathbf{x}) = \int \Pr(\mathbf{y}|\mathbf{x}, \phi) f(\phi) d\phi$$

- $f(\phi)$ effectively represents a large or infinite ensemble of models!

Dilution and Dropout

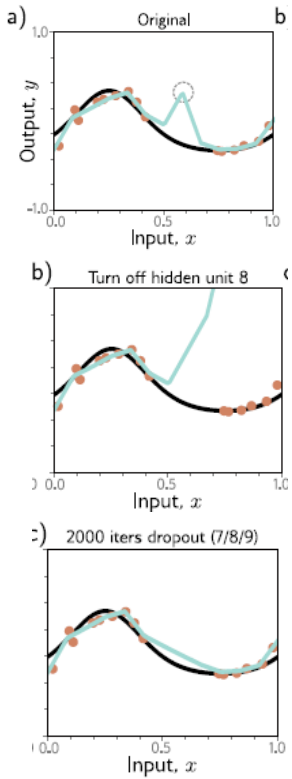
- **Dilution**: Weights are set to 0 with some probability during a training iteration
- Equivalent to removing random connections, although every hidden layer unit will typically still be able to pass its output to some node in the next layer

- **Dropout**: Set the *output* of random units to 0, encouraging the network to reduce strong dependence on any single unit
- In older networks, typical dropout rates may be as high as 50%!

- With dropout, we are simulating training a large ensemble of “thinned” networks with shared weights

Dropout and Regularization

- Dropout can be understood to encourage implicit regularization
- Suppose the network output has good performance on the data, but contains areas of roughness away from the data
- These regions do not affect loss, so difficult to discourage them
- With dropout, these regions may change substantially, affecting the outputs and thus the loss
- Network learns to reduce dependency on the corresponding units and output smoother functions



Prince

Adding Noise

- Dropout can also be interpreted as applying *noise* to the network activations
- Instead of zeroing them or the weights entirely, we can also add random noise to encourage less dependency on the weights

- Alternatively, we can add noise to input data or labels in a given iteration
- Due to maximum likelihood optimization, the network nominally tries to output large values for correct predictions and small values otherwise

- By randomly changing labels, we simulate the assumption that a training label is incorrect and belong to another class with equal probability

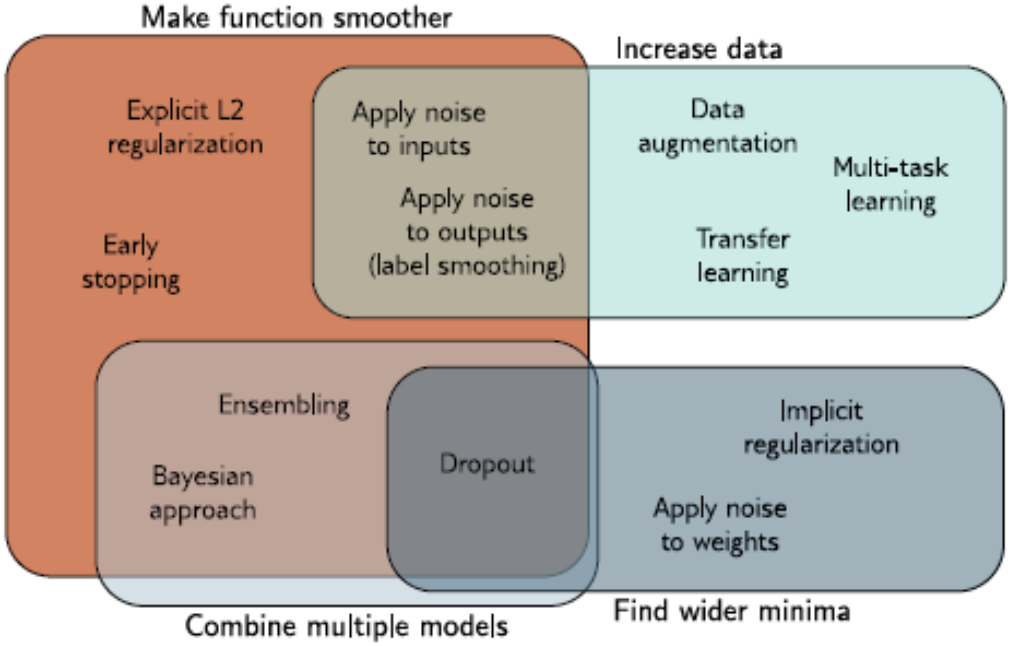
Data and Task Augmentation

- More training data can also reduce variance and increase model performance
- In *transfer learning*, we adapt a model already trained on another data set

- In some problems, we can augment the data using invariant transformations
- E.g., apply geometric or image transformations to a set of image data

- We can train a network to perform "auxiliary" tasks using *multi-task learning*
- Pushes the model to focus more on useful *representations* of the problem
- In *self-supervised learning*, auxiliary tasks are automatically generated from an unlabeled data set (e.g., mask parts of the input, apply transformations, etc.)

Summary



Prince